

GUIDE DU STAGIAIRE

DOCKER LA HAUTE MER

Swarm, Kubernetes, CI/CD et production. Vos consignes, concepts, commandes à exécuter, quiz et espaces de notes. Les corrigés sont présentés en séance.

5

MISSIONS

3

NŒUDS CLUSTER

1

GRAND PRIX



**SPARKS
FORMATION**

Guide stagiaire · Jour 3
Session inter · du 06/07/2026 au 08/07/2026
Docker Engine 29.x · Compose v2 · Swarm

La haute mer

Swarm, Kubernetes, CI/CD et production

Aujourd'hui, vous quittez la machine unique. Vous allez monter un vrai cluster et découvrir l'« état désiré » (Exercice 13), y déployer ShipFast avec des secrets et une mise à jour sans coupure (Exercice 14), traduire ces réflexes en vocabulaire Kubernetes, durcir votre stack pour la production (Exercice 15), automatiser vos builds en CI (Exercice 16), puis, en Grand Prix, tout refaire de zéro sur une application inconnue (Exercice 17) — la preuve que vos réflexes Docker sont acquis et transférables.

Au programme aujourd'hui

À la fin de cette journée, vous saurez :

- Monter un cluster Swarm de 3 nœuds et comprendre la notion d'état désiré.
- Déployer une application en production avec des secrets chiffrés et une mise à jour sans coupure (rolling update).
- Situer Swarm par rapport à Kubernetes et choisir le bon orchestrateur selon le contexte.
- Durcir une stack pour la production : ressources, logs, scan de sécurité, monitoring.
- Construire un pipeline CI/CD qui build, scanne et publie une image automatiquement.
- Mettre en production une application inconnue de A à Z, en autonomie.

HORAIRE	BLOC	ON CONSTRUIT QUOI ?
9h00 - 9h15	Quiz réveil + plan de journée	—
9h15 - 10h35	Exercice 13 — Monter un cluster Swarm 3 nœuds	Un cluster qui s'auto-répare
10h35 - 10h50	Pause	—
10h50 - 12h15	Exercice 14 — ShipFast en prod : stack, secrets, rolling update, chaos	Une prod scalée, mise à jour sans coupure
12h15 - 12h30	Débrief : état désiré, modèle mental des orchestrateurs	—
13h30 - 13h50	Et Kubernetes dans tout ça ? (kompose, k3d)	—
13h50 - 14h45	Exercice 15 — Durcir pour la production : ressources, logs, sécurité	La checklist prod, appliquée au fil rouge
14h45 - 15h20	Exercice 16 — L'usine à images : votre CI GitHub Actions	Un pipeline build + scan + push automatique
15h20 - 15h35	Pause	—
15h35 - 16h35	Exercice 17 — Le Grand Prix : capstone en binômes	Une app inconnue mise en prod, de zéro
16h35 - 17h00	Clôture, plan d'action, ressources, évaluations	—

9h00 — Quiz réveil (5 questions)

#	QUESTION	VOTRE RÉPONSE
1	Que se passe-t-il si la VM qui héberge votre Compose d'hier tombe ?	
2	Quel type de réseau Docker fonctionne entre plusieurs machines ?	
3	Un mot de passe dans <code>environment:</code> du compose : quel problème ?	
4	Citez les deux rôles d'un nœud Swarm.	
5	Hier, <code>--scale api=3</code> : qu'est-ce qui manquait pour appeler ça de la prod ?	

Les concepts du jour

Impératif vs déclaratif. Jusqu'ici, vous avez piloté Docker de façon impérative : vous dites précisément quoi faire (« lance ce conteneur », « scale à 3 »). Un orchestrateur fonctionne de façon déclarative : vous décrivez un **état désiré** (« je veux 3 réplicas de nginx, toujours ») et un contrôleur (le manager Swarm, ou le control-plane Kubernetes) se charge en permanence de faire converger la réalité vers cet état — y compris après une panne. Vous ne dites plus « quoi faire », vous dites « quoi maintenir ».

À RETENIR

Un cluster Swarm est composé de **managers** (ils décident, maintiennent l'état désiré) et de **workers** (ils exécutent les conteneurs). Le **routing mesh** fait qu'un port publié répond sur tous les nœuds du cluster, même ceux qui n'hébergent aucun réplica du service : chaque nœud route la requête via le réseau overlay vers un réplica sain, avec répartition de charge intégrée.

La mise à jour sans coupure (rolling update). Le cœur de la production, c'est de pouvoir déployer une nouvelle version d'une application sans interrompre le service. Deux ingrédients rendent cela possible :

- **update_config: order: start-first** : le nouveau réplica démarre *avant* que l'ancien soit arrêté (au lieu de couper puis relancer).
- **Un healthcheck** : l'orchestrateur ne bascule le trafic vers le nouveau réplica qu'une fois celui-ci déclaré « healthy ». Sans healthcheck, un réplica qui démarre mais ne répond pas encore peut recevoir du trafic dans le vide.

À RETENIR

Le triangle de la production, valable pour Swarm, Kubernetes, ECS ou Cloud Run : ① **une image dans un registry** (jamais construite localement sur le serveur cible), ② **un état désiré déclaré** (réplicas, ressources, politique de mise à jour), ③ **des healthchecks** sur chaque service. Ces trois piliers sont la base de tout déploiement fiable.

Choisir son orchestrateur. Il n'y a pas une seule bonne réponse — cela dépend de l'échelle et du contexte :

- **1 VM et moins de 10 services** → Docker Compose reste parfaitement légitime, à condition d'avoir de bons healthchecks, un `restart: unless-stopped` et des sauvegardes testées.
- **2 à 5 VM, équipe réduite** → Swarm ou K3s (une distribution légère de Kubernetes).

- **Besoin d'écosystème** (autoscaling, opérateurs, cloud managé, recrutement facilité) → Kubernetes, via un service managé (EKS, AKS, GKE) ou souverain (OVHcloud MKS, Scaleway Kapsule).
- **Pas d'orchestrateur du tout** → les conteneurs serverless managés (Cloud Run, ECS Fargate, Azure Container Apps) : vous fournissez une image, la plateforme gère le reste.

Le pont conceptuel entre Swarm et Kubernetes : un *service* Swarm correspond à un *Deployment* + *Service* K8s ; une *stack* correspond à des manifestes ou un chart Helm ; le *routing mesh* correspond à kube-proxy/Ingress ; un *secret* Swarm correspond à un *Secret* K8s ; un *nœud en drain* correspond à un *node cordon* + *drain*. Si vous maîtrisez les concepts de ce matin, vous avez déjà l'essentiel de Kubernetes en tête — il ne reste que du vocabulaire et du YAML en plus.

La sécurité en deux temps. Sécuriser des conteneurs se joue à deux moments distincts, et il faut les deux :

- **Avant l'exécution** : scanner l'image pour détecter les vulnérabilités connues et les mauvaises pratiques (Trivy, Grype, Docker Scout, hadolint, détection de secrets, SBOM).
- **Pendant l'exécution** : détecter un comportement anormal en temps réel sur un conteneur qui tourne (Falco, qui écoute les appels système du noyau via eBPF).

À RETENIR

Une image dont le scan est parfaitement propre peut être compromise à l'exécution par une faille applicative ou une injection : un scanner d'image ne le verra jamais, un outil de détection runtime (Falco) le peut. Le couple **scan d'image + détection runtime** est le socle de sécurité de référence pour des conteneurs en production.

EXERCICE 13 · 9H15-10H35 · 80 MIN

Monter un cluster Swarm 3 nœuds

Objectif : monter un cluster Swarm de 3 nœuds sur Play with Docker et observer un service qui répartit sa charge, encaisse une panne et se répare seul.

CE QUE VOUS DEVEZ FAIRE

Sur **labs.play-with-docker.com**, connectez-vous avec votre Docker ID et créez 3 instances. Initialisez le cluster sur la première (le futur manager), faites rejoindre les deux autres, puis suivez les étapes ci-dessous : créez un service répliqué, observez le routing mesh, scalez-le, tuez un conteneur pour observer l'auto-réparation, puis mettez un nœud en maintenance sans coupure de service.

```
# Chacun : labs.play-with-docker.com → login Docker ID → "+ ADD NEW INSTANCE" x3
# — Sur node1 (le futur manager) —
docker swarm init --advertise-addr eth0
# → copier la commande "docker swarm join --token SWMTKN-1-... IP:2377" affichée

# — Sur node2 et node3 : coller la commande join —

# — Retour sur node1 —
docker node ls                                # 3 nœuds, node1 Leader

# 1. Premier service répliqué
```

```

docker service create --name web --replicas 3 -p 8080:80 nginx:1.30-alpine
docker service ls
docker service ps web          # → répartis sur les 3 nœuds

# 2. La magie du routing mesh : le port 8080 répond sur LES TROIS nœuds,
# même celui qui n'héberge aucun réplica (cliquez les badges "8080" de PWD).
# Q1 : comment est-ce possible ?

# 3. Scaling en une commande
docker service scale web=6 && docker service ps web

# 4. Auto-réparation (le moment "waouh")
# Sur node3 : docker ps → tuez un conteneur : docker rm -f <id>
# Sur node1 : docker service ps web
# Q2 : que constatez-vous ? Qui a agi, et pourquoi ?

# 5. Maintenance d'un nœud sans coupure
docker node update --availability drain node3
docker service ps web          # tout a migré sur node1/node2
docker node update --availability active node3

# 6. BONUS : trouvez la commande qui montre, en continu,
# la convergence d'un service (indice : docker service ps ne rafraîchit pas...
# mais watch existe, et docker service logs aussi).

```

Questions à préparer :

- Q1 : comment le port répond-il sur les trois nœuds, même sans réplica local ?
- Q2 : qui a recréé le conteneur détruit, et selon quelle logique ?
- Q3 : que change `docker node update --availability drain` pour le service en cours ?

ATTENTION

Si vous collez la commande `join` sur node1 (déjà manager), vous obtiendrez une erreur. Pensez aussi à ne pas oublier `--advertise-addr eth0` à l'initialisation, sinon Swarm peut hésiter entre deux interfaces réseau. Enfin, les sessions Play with Docker expirent : sauvegardez vos commandes dans un fichier local, pas uniquement dans le terminal web.

NOTES

EXERCICE 14 · 10H50-12H15 · 85 MIN

ShipFast en prod : stack, secrets, rolling update

Objectif : déployer ShipFast sur votre cluster avec une image poussée sur un registry, un mot de passe en secret chiffré, une API scalée à 4 réplicas, et réaliser une mise à jour applicative sans perdre une seule requête.

CE QUE VOUS DEVEZ FAIRE

Toujours sur le cluster Play with Docker de l'exercice 13. Récupérez le fil rouge, construisez et poussez votre image, créez un secret Docker pour le mot de passe, déployez la stack fournie, scalez l'API, puis réalisez une mise à jour de version en observant une sonde de disponibilité qui ne doit afficher que des réponses 200 pendant toute la bascule.

```
# — Sur node1 —
# 1. Récupérer le fil rouge (dépôt préparé en amont) et se connecter à Docker Hub
git clone https://github.com/<votre-compte>/formation-docker.git && cd formation-docker/
shipfast
docker login # le Docker ID créé pour PWD sert enfin !

# 2. Builder et pousser (un cluster tire ses images d'un registry, jamais du disque)
docker build -t <dockerid>/shipfast:v3 .
docker push <dockerid>/shipfast:v3

# 3. Le mot de passe sort du YAML : secret chiffré dans le cluster
echo "formation-${hostname}" | docker secret create pg_password -
docker secret ls # présent...
docker secret inspect pg_password # ...mais illisible. C'est le but.

# 4. Déployer la stack (fichier stack.yaml fourni ;
# remplacer <dockerid> par le vôtre)
docker stack deploy -c stack.yaml ship
docker stack services ship # attendre les REPLICAS à x/x
curl -X POST localhost:8080/links -H "Content-Type: application/json" \
  -d '{"url":"https://kubernetes.io"}'

# 5. Scaler l'API sous les yeux du routing mesh
docker service scale ship_api=4
for i in $(seq 1 8); do curl -s localhost:8080/stats | grep -o '"host": "[^"]*"'; done
# → 4 hostnames différents : la répartition de charge, prouvée en une boucle.

# 6. ROLLING UPDATE SANS COUPURE
# a. Modifier server.js : passez VERSION à "v4" (ligne 1)
# b. docker build -t <dockerid>/shipfast:v4 . && docker push <dockerid>/shipfast:v4
# c. Dans un 2e onglet node2, la sonde de disponibilité :
while true; do curl -s -o /dev/null -w "%{http_code} " localhost:8080/health; sleep 0.5; done
# d. Sur node1, la mise à jour :
docker service update --image <dockerid>/shipfast:v4 ship_api
# → observez la sonde pendant toute la bascule.
# e. Gardez aussi l'interface web ouverte (http://nodeX:8080) pour voir
# le badge de version changer en direct.

# 7. Et si la v4 était pourrie ?
docker service rollback ship_api
docker service ps ship_api # l'historique des bascules est lisible

# 8. BONUS chaos : sur node1, docker node update --availability drain node2
# pendant que la sonde tourne. La prod doit rester à 200. Prouvez-le.
```

Questions à préparer :

- Que constatez-vous sur la sonde de disponibilité pendant la mise à jour ? Pourquoi ce résultat est-il possible ?
- À quoi sert `order: start-first` dans la configuration de mise à jour d'un service ?
- Pourquoi le mot de passe passe-t-il par un secret Docker plutôt que par une variable d'environnement du YAML ?
- Pourquoi la base de données est-elle contrainte à un nœud précis (`placement: constraints`) alors que l'API ne l'est pas ?

DÉFI BONUS

Réalisez l'étape 8 : drainez un nœud pendant que la sonde de disponibilité tourne, et prouvez que la production reste à 200 tout du long.

NOTES

13h30 — Et Kubernetes dans tout ça ?

Démonstration : sur un cluster K8s local créé en 30 secondes (`k3d cluster create demo`), le `compose.yaml` d'hier est converti en manifestes Kubernetes avec `kompose convert -f compose.yaml`, puis appliqué avec `kubectll apply -f .` et suivi avec `kubectll get pods -w`. Un aperçu de `k9s` (l'équivalent de lazydocker pour Kubernetes) vous est également montré.

EXERCICE 15 · 13H50-14H45 · 55 MIN

Durcir pour la production : ressources, logs, sécurité

Objectif : transformer votre `compose.yaml` d'hier en version production-ready et repartir avec la checklist prod complète.

CE QUE VOUS DEVEZ FAIRE

Retour sur vos postes locaux, sur le `compose.yaml` d'hier. Enchaînez les trois ateliers ci-dessous : limiter les ressources d'un conteneur, faire la rotation des logs, puis scanner vos images avec Trivy.

Atelier 1 — Brider les ressources

```
# Constatez le danger : un conteneur gourmand SANS limite
docker run --rm -it --name glouton polinux/stress \
  stress --vm 1 --vm-bytes 512M --timeout 10s      # docker stats à côté : ça grimpe

# La ceinture de sécurité : mêmes appétits, plafond à 256 Mo
docker run --rm -it -m 256m --memory-swap 256m polinux/stress \
  stress --vm 1 --vm-bytes 512M --timeout 10s

# Q1 : que se passe-t-il cette fois ? Que devient le conteneur, que devient l'hôte ?

# À reporter dans votre compose (service api) :
#   mem_limit: 256m
```

```
#      cpus: 0.5
#      restart: unless-stopped
```

Atelier 2 — Les logs qui remplissent les disques

```
# Repérez le fichier de logs d'un conteneur
docker inspect shipfast-api-1 --format '{{.LogPath}}'

# La parade, à ajouter service par service dans le compose :
#   logging:
#     driver: json-file
#     options: { max-size: "10m", max-file: "3" }

# Validation :
docker compose up -d
docker inspect shipfast-api-1 --format '{{.HostConfig.LogConfig}}'
```

Atelier 3 — Scanner ses images avec Trivy

```
# Trivy en un conteneur, 3 usages en 3 commandes
# (le cache de la base CVE est monté pour ne pas la retélécharger à chaque fois) :
alias trivy='docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  -v trivy-cache:/root/.cache/ aquasec/trivy:0.69.3' # tag FIXE, jamais latest

# 1) Vulnérabilités (CVE) – le cas d'usage vedette
trivy image --severity HIGH,CRITICAL localhost:5000/shipfast:v3
# 2) Secrets oubliés dans l'image (clés, tokens, .env)
trivy image --scanners secret localhost:5000/shipfast:v1
# 3) Mauvaises pratiques du Dockerfile (root, latest, apt non figé...)
trivy config .

# Exercice : partez d'une image volontairement vieille (ex. node:18), scannez,
# comptez les CRITICAL, passez à node:24-alpine, re-scannez.
# Q2 : combien de CRITICAL en moins entre les deux images de base ?
```

Pour compléter votre vision de la production : **Dozzle** (tous les logs en web temps réel) et **cAdvisor** (métriques CPU/RAM/réseau) donnent une visibilité immédiate sur une stack qui tourne. Côté sécurité runtime, **Falco** (CNCF) écoute les appels système du noyau et alerte en temps réel si un comportement anormal se produit — par exemple l'ouverture d'un shell interactif dans un conteneur de production, souvent le signe qu'un attaquant a pris pied dedans.

LA CHECKLIST PROD — À CONSERVER

Tag d'image épinglé (jamais `latest`) et image scannée (Trivy/Scout) — utilisateur non-root — healthcheck sur chaque service — `restart: unless-stopped` (Compose) ou `restart_policy` (Swarm) — limites mémoire/CPU — rotation des logs — secrets hors du YAML et hors de l'image — volumes nommés + sauvegarde testée — réseau interne pour les données — un seul port publié, derrière un reverse proxy — supervision (logs + métriques) — mises à jour d'images suivies (par exemple avec Diun ou Renovate).

Questions à préparer :

- Q1 : quel comportement observez-vous une fois la limite mémoire appliquée, et pourquoi est-ce le comportement souhaité ?

- Q2 : quel est l'écart de vulnérabilités critiques entre l'image de base ancienne et l'image alpine récente ?
- Quelle est la différence entre ce que détecte Trivy et ce que détecte Falco ?

NOTES

EXERCICE 16 · 14H45-15H20 · 35 MIN

L'usine à images : votre CI GitHub Actions

Objectif : mettre en place un pipeline qui, à chaque push, construit l'image, la scanne avec Trivy et la publie automatiquement sur un registry (GHCR).

CE QUE VOUS DEVEZ FAIRE

Créez un dépôt GitHub `shipfast` avec les fichiers du fil rouge (Dockerfile v2, `server.js`, `package.json`, `.dockerignore`). Ajoutez le workflow ci-dessous dans `.github/workflows/docker.yml`, poussez, et observez l'onglet Actions travailler pour vous. Livrable : le lien vers votre run vert et votre image visible dans l'onglet Packages.

```
# .github/workflows/docker.yml
name: build-scan-push
on:
  push:
    branches: [main]

jobs:
  docker:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v6

      - name: Lint du Dockerfile
        uses: hadolint/hadolint-action@v3.1.0

      - uses: docker/setup-buildx-action@v3

      - name: Login GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${GITHUB_ACTOR}
          password: ${GITHUB_TOKEN}

      - name: Build (sans push) pour le scan
        uses: docker/build-push-action@v6
        with:
          context: .
```

```

load: true
tags: ghcr.io/${{ github.repository }}:${{ github.sha }}

- name: Scan Trivy – bloque le pipeline si CRITICAL
  uses: aquasecurity/trivy-action@v0.36.0 # version épinglée et signée – jamais
@master
  with:
    image-ref: ghcr.io/${{ github.repository }}:${{ github.sha }}
    severity: CRITICAL
    exit-code: "1"

- name: Push (seulement si le scan est passé)
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: |
      ghcr.io/${{ github.repository }}:${{ github.sha }}
      ghcr.io/${{ github.repository }}:latest
    cache-from: type=gha
    cache-to: type=gha,mode=max

```

Questions à préparer :

- Ce pipeline reprend des briques déjà vues les jours précédents : lesquelles reconnaissez-vous (lint, build avec cache, registry + tag, scan) ?
- Pourquoi tagger l'image avec le SHA du commit répond-il à la question « pourquoi jamais `latest` en prod » ?
- Que se passe-t-il si le scan Trivy détecte une vulnérabilité CRITICAL ?
- Comment prolongeriez-vous ce pipeline pour aller jusqu'au déploiement (CD) ?

NOTES

EXERCICE 17 · 15H35-16H35 · 60 MIN · LE GRAND PRIX

Capstone en binômes : une application inconnue en production

Objectif : mettre en production, en binôme et en autonomie complète, une application que vous découvrirez à l'instant — la preuve finale que vos réflexes Docker sont universels.

LE DÉFI

Vous recevez `meteo-du-port/` : une petite API Python/Flask qui donne la météo marine, avec un cache Redis. Aucun Dockerfile n'est fourni. Vous n'avez peut-être jamais fait de Python — c'est justement l'occasion de prouver que vos réflexes Docker sont transférables à n'importe quel langage. Documentation et notes personnelles autorisées.

Livrables attendus en 55 minutes :

- Une image optimisée : multi-stage, non-root, moins de 200 Mo.

- Un `compose.yaml` complet : API + Redis, réseau dédié, healthchecks, limites de ressources, rotation des logs, restart policy.
- Un scan Trivy sans vulnérabilité CRITICAL.
- L'application qui répond sur `curl localhost:8090/meteo/brest`, avec un cache fonctionnel (le 2^e appel est instantané, avec `"source": "cache"`).
- Une démonstration de 3 minutes devant l'autre binôme.

CRITÈRE	POINTS
L'application fonctionne (curl OK, cache Redis actif — 2 ^e appel instantané)	6
Dockerfile : multi-stage, image slim/alpine, non-root, .dockerignore	6
Compose : healthchecks + depends_on conditionnel + réseau dédié	5
Prod-ready : limites ressources + rotation logs + restart policy	4
Scan Trivy sans CRITICAL (rapport à l'écran)	2
Qualité de la démo (clarté, choix expliqués)	2

ATTENTION

Pensez à définir la variable de connexion à Redis attendue par l'application (souvent une variable du type `REDIS_URL`) — sans elle, le cache ne fonctionnera pas même si les deux conteneurs tournent. Ne publiez pas le port de Redis vers l'extérieur : seule l'API doit être exposée.

NOTES

Les outils IA (2026)

OBJECTIF

L'IA est un **copilote, pas le pilote** : elle propose, vous validez, testez et scannez. Voici les outils à connaître et les règles de sécurité qui vont avec.

Gordon est l'agent IA intégré nativement à Docker, accessible en ligne de commande (`docker ai`) et dans Docker Desktop. Il a accès au CLI Docker, au shell et aux fichiers du projet courant. Il fonctionne en **human-in-the-loop** : il affiche l'action qu'il veut exécuter, vous approuvez ou refusez.

```
# Lancer l'agent en TUI conversationnelle
docker ai

# Exemples de prompts utiles :
# « Why won't my container start? » → il inspecte logs + statut, propose un fix
# « Optimize this Dockerfile for a smaller image »
```

```
# « Containerize this Node.js project for me »  
# « List all running containers, then stop the one using the most memory »
```

`docker init` est une commande officielle qui génère d'un coup `Dockerfile`, `compose.yaml`, `.dockerignore` et un `README.Docker.md`. Docker détecte le langage de votre projet (package.json, requirements.txt, go.mod, Cargo.toml, etc.), pose quelques questions, et produit un Dockerfile multi-stage déjà propre.

```
$ docker init  
? What application platform does your project use? Node  
? What version of Node do you want to use? 24  
? Which package manager do you want to use? npm  
? What command to start the app? node server.js  
? What port does your server listen on? 3000  
  
CREATED: .dockerignore  
CREATED: Dockerfile  
CREATED: compose.yaml  
CREATED: README.Docker.md  
  
$ docker compose up --build # l'app tourne
```

GitHub Copilot propose l'agent `@docker` dans Copilot Chat (VS Code et github.com). Il génère les fichiers Docker adaptés à votre projet, explique les concepts, et peut — avec votre accord — ouvrir une pull request contenant les fichiers générés.

```
# Dans Copilot Chat :  
@docker What does containerizing an application mean?  
@docker How would I containerize this project?  
@docker how can I start a container with a volume?  
@docker generate a Dockerfile and open a PR
```

Docker Scout analyse en continu vos images (CVE, SBOM) et — c'est là que l'IA intervient — hiérarchise et propose les correctifs : quelle mise à jour d'image de base élimine le plus de vulnérabilités pour le moindre effort.

```
# Lister les CVE, puis demander la meilleure action corrective  
docker scout cves shipfast:v2  
docker scout recommendations shipfast:v2  
  
# Cibler ce qui est réellement exploitable (score EPSS)  
docker scout cves --epss-score 0.5 --only-severity critical,high shipfast:v2
```

Là où Trivy liste les failles, Scout les priorise et propose l'action : la différence entre « 200 CVE » (paralysant) et « changez votre base pour `node:24-alpine`, vous en effacez 180 » (actionnable).

LES 4 RÈGLES DE SÉCURITÉ IA

1. Jamais de secret dans un prompt. Tout ce que vous tapez à Gordon ou Copilot peut transiter par des services tiers. **2. L'IA hallucine** parfois des tags d'images et des flags qui n'existent pas. Vérifiez toujours chaque tag et chaque paquet avant de les utiliser. **3. Relisez et scannez TOUJOURS** le Dockerfile généré : pas de root, pas de secret en clair dans un ENV, pas d'ADD d'une URL distante, image de base à jour — puis passez `docker scout cves`. **4. Méfiez-vous des données « anodines ».** Un contenu piégé dans une image (par exemple un LABEL) peut chercher à injecter des instructions à un agent IA : ne faites jamais confiance aveuglément à une source externe.

MINI-TP IA · ~25 MIN

Le copilote à l'épreuve

Objectif : installer le réflexe « générer → relire → corriger → scanner » face à un Dockerfile produit par une IA.

CE QUE VOUS DEVEZ FAIRE

Repartez d'une application inconnue (le dossier `meteo-du-port` du Grand Prix). Laissez l'IA échafauder un Dockerfile, relisez-le en répondant aux questions ci-dessous, construisez l'image, puis scannez-la. Si votre poste n'a pas Docker Desktop avec Gordon activé, utilisez la variante `docker init` + Scout, qui ne nécessite pas de compte.

```
# 1. Repartir d'une app inconnue
cd ~/formation-docker/meteo-du-port

# 2. Laisser l'IA échafauder
docker init                # ou : docker ai « containerize this app »

# 3. RELIRE le Dockerfile généré et répondre par écrit :
# Q1 : tourne-t-il en root ? multi-stage ? un .dockerignore a-t-il été créé ?
# Q2 : le tag de base est-il épinglé (pas latest) ?

# 4. Construire, puis SCANNER — ne jamais faire confiance sans vérifier
docker build -t meteo:ia .
docker scout cves meteo:ia
docker scout recommendations meteo:ia
# Q3 : quelle recommandation appliquer en premier, et pourquoi (EPSS) ?
```

Questions à préparer :

- Le Dockerfile généré tourne-t-il en root ou en utilisateur non privilégié ?
- Le tag de l'image de base est-il épinglé, ou risque-t-il de changer silencieusement ?
- Quelle recommandation Scout appliqueriez-vous en premier, et pourquoi ?
- Qu'est-ce que ce mini-TP change dans votre façon d'utiliser un assistant IA pour écrire du Dockerfile ?

Quiz de clôture — synthèse des 3 jours

#	QUESTION	VOTRE RÉPONSE
1	Quelle est la différence entre une approche impérative et une approche déclarative pour gérer des conteneurs ?	
2	Citez les deux ingrédients qui rendent possible une mise à jour sans coupure.	
3	Quels sont les trois piliers du triangle de la production ?	
4	Dans quel cas choisiriez-vous Compose plutôt qu'un orchestrateur ?	
5	Quelle est la différence entre un scan d'image (Trivy) et une détection runtime (Falco) ?	
6	Citez trois éléments de la checklist production.	
7	Pourquoi tagger une image avec le SHA d'un commit en CI plutôt qu'avec <code>latest</code> ?	
8	Quelles sont les 4 règles de sécurité à respecter en utilisant un assistant IA pour générer du Dockerfile ?	
9	Jour 1 : pourquoi éviter de faire tourner un conteneur en root ?	
10	Jour 2 : quelle est la différence entre un volume nommé et un bind mount ?	
11	Jour 3 : qu'apporte un healthcheck lors d'un déploiement ?	
12	Quelle est la première chose que vous allez conteneuriser ou améliorer une fois de retour au travail ?	

NOTES ET RESSOURCES POUR CONTINUER